

Universidad Mariano Galvez de Guatemala.
Facultad de Ingeniería en Sistemas de Información y Ciencias de la
Computación.
Curso: 021- Métodos Numéricos
Ing. MA. Carmelo Estuardo Mayén Monterroso



MANUAL TÉCNICO
Plataforma de Ejecución, Trazabilidad y Comparación de Métodos Numéricos
con Arquitectura Híbrida

Nombres:

César Gustavo Castillo García

Luis Enrique Escobar vides

Eduardo Leonel Monzón López

Bayrón Esau Morales Mazariegos

José Adrián Monterroso Melgar

Nathalie María Amalia Carbajal García

Carné:

1790-24-20456

1790-24-26001

1790-24-8668

1790-24-26929

1790-24-14368

1790-24-15648

Guatemala, 27 de mayo de 2026

INDICE

1. Descripción detallada de la arquitectura del sistema.	4
1.1 Flujo asíncrono detallado	5
1.2 Características clave de la arquitectura hibrida	5
1.2.1 Desacoplamiento real.....	5
1.2.2 Idempotencia y consistencia de estados	6
1.2.3 Contenerización completa	6
1.2.4 Pasos para realizar la Contenerización Completa	6
1.3 Justificación de la arquitectura hibrida en el contexto del proyecto.	10
2. Diagrama C4 (Context + Container) o arquitectura del sistema	10
2.1 Diagrama de contexto	10
2.2 Diagrama de contenedores.....	11
2.3 Diagrama de Componentes (API. NET)	11
2.4 Tabla de contenedores.....	12
3. Justificación de decisiones técnicas.....	13
3.1. ¿Por qué Redis?	13
3.2 ¿Por qué Python para el worker?	13
3.3 ¿Por qué Arquitectura Hibrida?.....	13
4. Modelo de base de datos	14
4.1 Script de Creación Completo	14
4.2 Diagrama Entidad-Relación	16
4.3 Tabla JOBS (Entidad Principal).....	17
4.4 Tabla JOBITERATIONS (Trazabilidad)	18
4.5 Relaciones y Restricciones.....	19
4.6 Índices para la optimización de rendimiento	19
4.7 Ciclo de vida de los Estados.....	19
5. Instrucciones para clonar el repositorio.....	20
5.1 Herramientas que necesitaremos.	20
5.2 Instrucciones.....	20
6. Guia de configuración de variables de entorno	21
6.1 Archivo de plantilla (.env.example)	22
6.2 Configuración Paso a Paso	22
6.3 Como se inyectan las variables en cada servicio API .NET.....	22

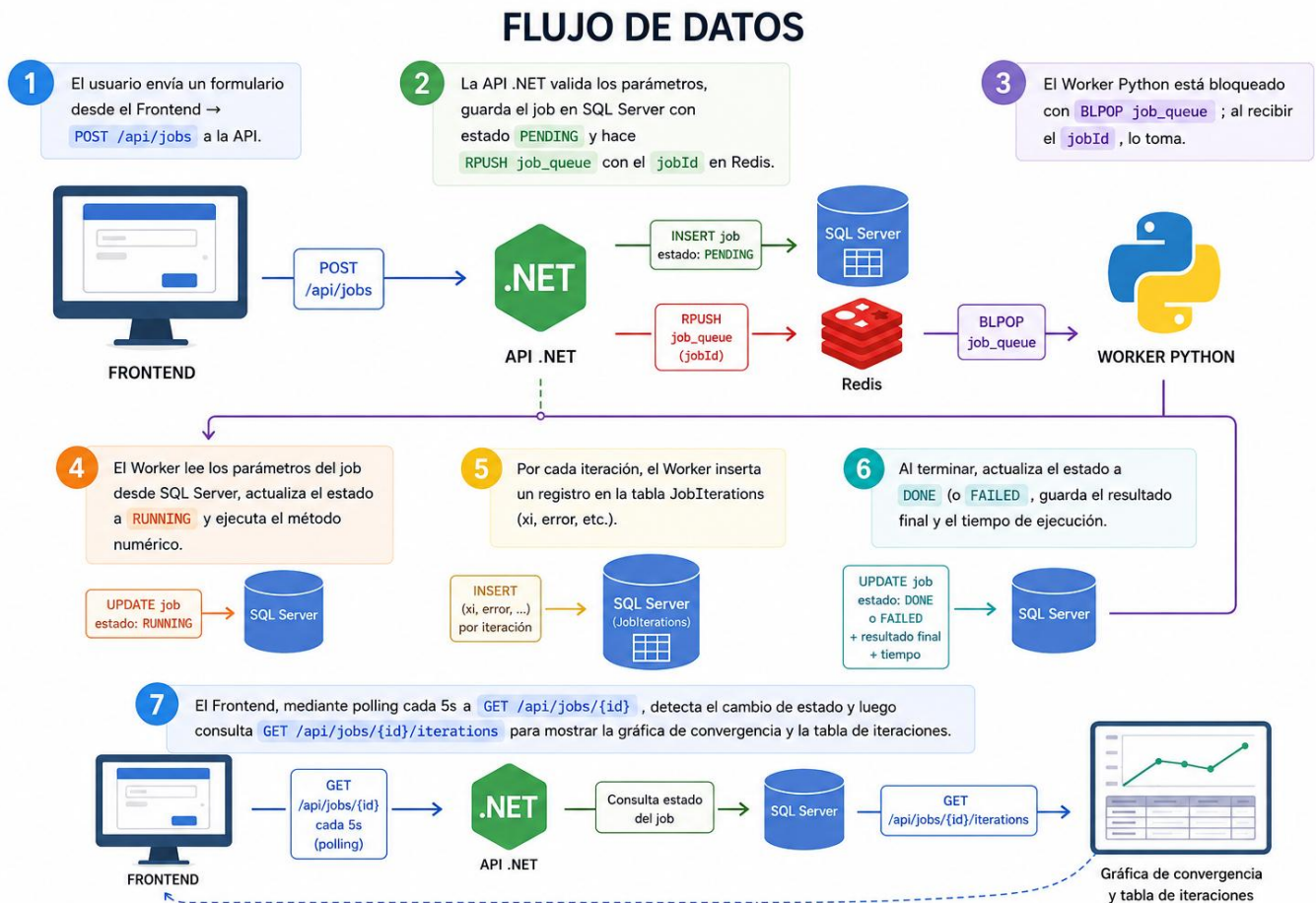
7. Guía Paso a Paso para levantar el sistema con Docker	24
7.1 Requisitos previos	24
7.2 Solución a problemas comunes.....	25
8. Pruebas realizadas y casos de error documentados con evidencia (Capturas o logs)	26

1. Descripción detallada de la arquitectura del sistema.

APLIMETODOS es una plataforma web distribuida de código abierto que resuelve métodos numéricos de forma asíncrona, registrando cada iteración y permitiendo la comparación de algoritmos. Su arquitectura se compone de cinco servicios independientes contenerizados, orquestados con Docker Compose.

Componente	Tecnología	Rol
 Frontend SPA	React 19 + TypeScript + Vite + Tailwind + Recharts + KaTeX	Interfaz de usuario: formularios, polling cada 5s, gráficas de convergencia y tabla de interacciones.
 API Principal	.NET 10 + EF Core + StackExchange.Redis	Endpoints REST, validación de parámetros, persistencia en SQL Server, encolado en Redis (<code>RPUSH</code>).
 Worker Numérico	Python 3.12 + SymPy + NumPy + pyodbc	Consume la cola (<code>BLPOP</code>), ejecuta los 6 métodos numéricos, guarda cada iteración en BD.
 Cola Redis	Redis 7	Lista <code>job_queue</code> para desacoplar la API del worker.
 Base de Datos	SQL Server 2022	Tablas <code>Jobs</code> y <code>JobIterations</code> para persistencia de jobs e iteraciones.

1.1 Flujo asíncrono detallado



1.2 Características clave de la arquitectura híbrida

1.2.1 Desacoplamiento real

- La API y el Worker se comunican exclusivamente a través de Redis y la base de datos. No hay llamadas HTTP directas del Worker a la API. Esto permite:
- Escalar workers horizontalmente (ejecutar múltiples contenedores de Python en paralelo) sin modificar la API.
- Someter la API a carga de peticiones ligeras mientras el Worker realiza cómputo pesado.

1.2.2 Idempotencia y consistencia de estados

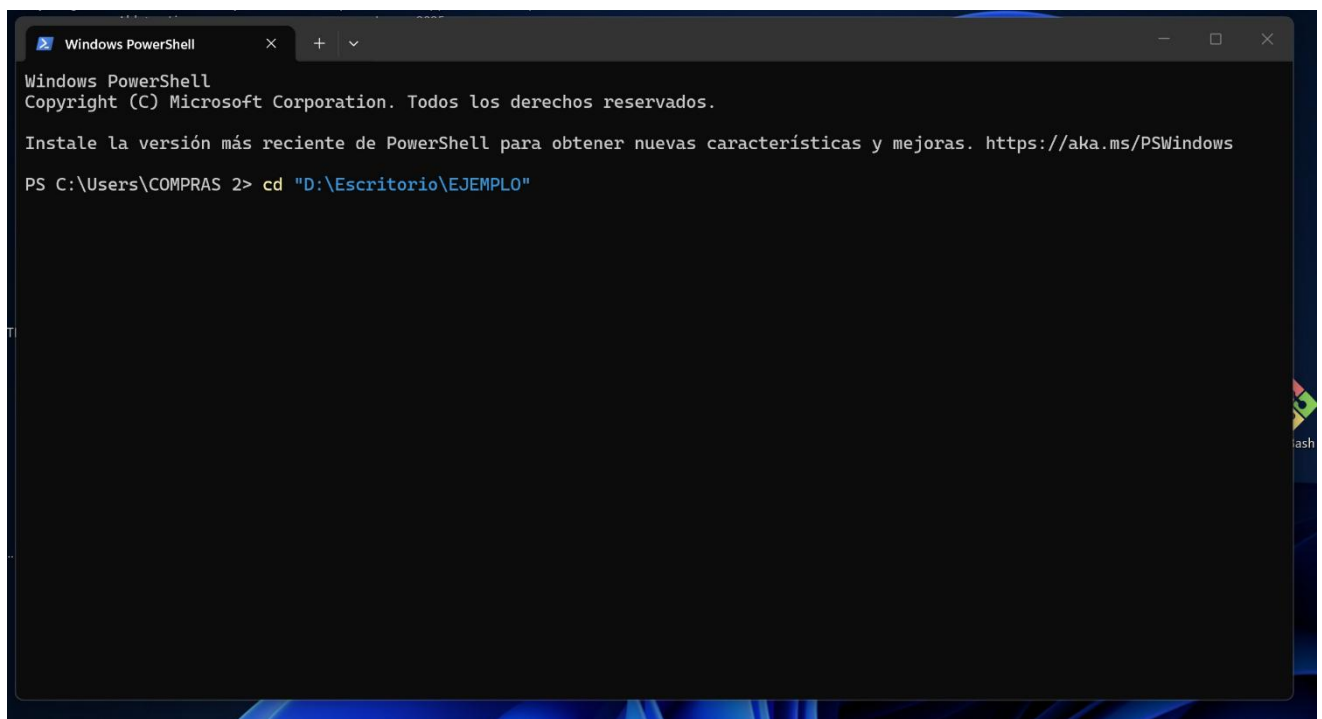
El Worker, antes de ejecutar un job, verifica que el estado en BD sea PENDING. Si un job ya está RUNNING o DONE (por ejemplo, por un reintento accidental desde la cola), el Worker lo descarta. Esto garantiza que no haya ejecuciones duplicadas.

1.2.3 Contenerización completa

El sistema se levanta con un solo comando: `docker compose up --build`. Todos los servicios (incluyendo la base de datos y Redis) se inician con la configuración correcta, sin pasos manuales adicionales. Esto asegura reproducibilidad en cualquier entorno.

1.2.4 Pasos para realizar la Contenerización Completa

- Primero ingresar a la carpeta donde se va a clonar el repositorio.

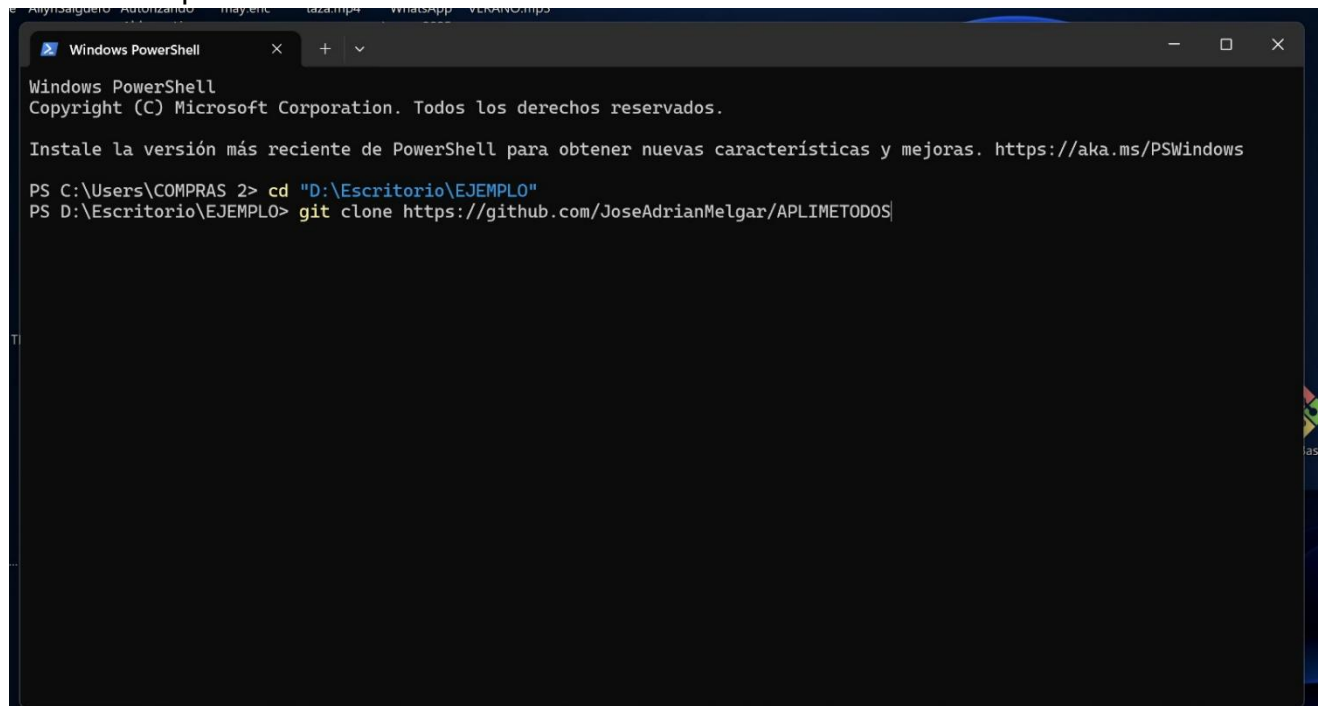


```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\COMPRAS 2> cd "D:\Escritorio\EJEMPLO"
```

- Dentro de la carpeta, escribir el comando git clone seguido del link del repositorio

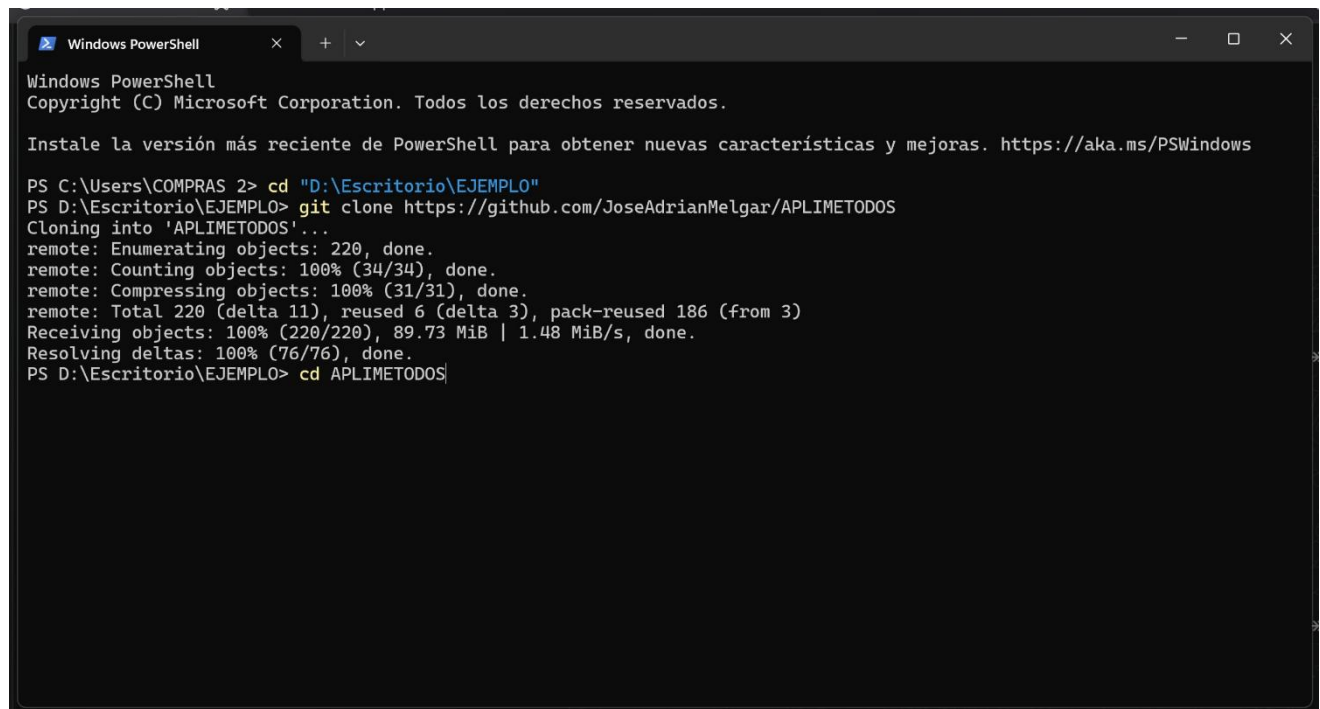


```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\COMPRAS 2> cd "D:\Escritorio\EJEMPLO"
PS D:\Escritorio\EJEMPLO> git clone https://github.com/JoseAdrianMelgar/APLIMETODOS
```

- Una vez clonado el repositorio, ingresar la carpeta APLIMETODOS



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\COMPRAS 2> cd "D:\Escritorio\EJEMPLO"
PS D:\Escritorio\EJEMPLO> git clone https://github.com/JoseAdrianMelgar/APLIMETODOS
Cloning into 'APLIMETODOS'...
remote: Enumerating objects: 220, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (31/31), done.
remote: Total 220 (delta 11), reused 6 (delta 3), pack-reused 186 (from 3)
Receiving objects: 100% (220/220), 89.73 MiB | 1.48 MiB/s, done.
Resolving deltas: 100% (76/76), done.
PS D:\Escritorio\EJEMPLO> cd APLIMETODOS
```

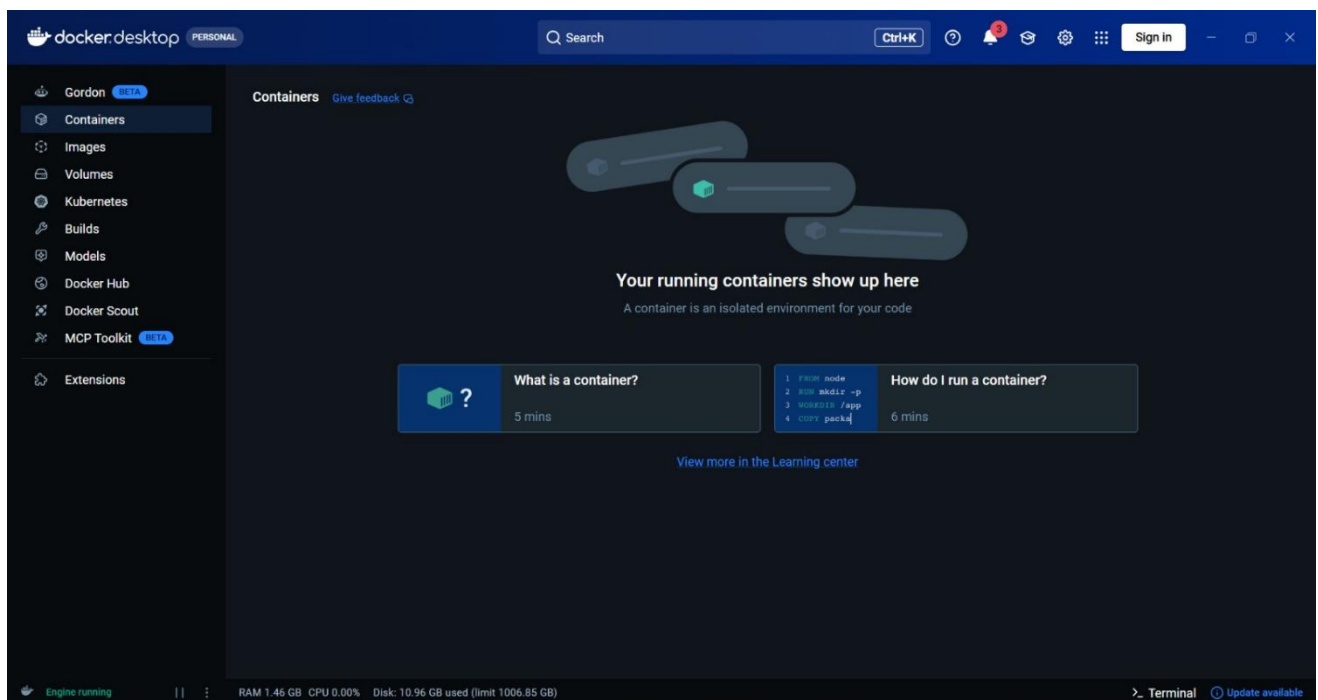
- Dentro de la carpeta, copiamos el archivo .env con el comando `copy .env.example .env`

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\COMPRAS 2> cd "D:\Escritorio\EJEMPLO"
PS D:\Escritorio\EJEMPLO> git clone https://github.com/JoseAdrianMelgar/APLIMETODOS
Cloning into 'APLIMETODOS'...
remote: Enumerating objects: 220, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (31/31), done.
remote: Total 220 (delta 11), reused 6 (delta 3), pack-reused 186 (from 3)
Receiving objects: 100% (220/220), 89.73 MiB | 1.48 MiB/s, done.
Resolving deltas: 100% (76/76), done.
PS D:\Escritorio\EJEMPLO> cd APLIMETODOS
PS D:\Escritorio\EJEMPLO\APLIMETODOS> copy .env.example .env
```

- Abrir docker y asegurarse que muestre Engine Running en la esquina inferior izquierda.



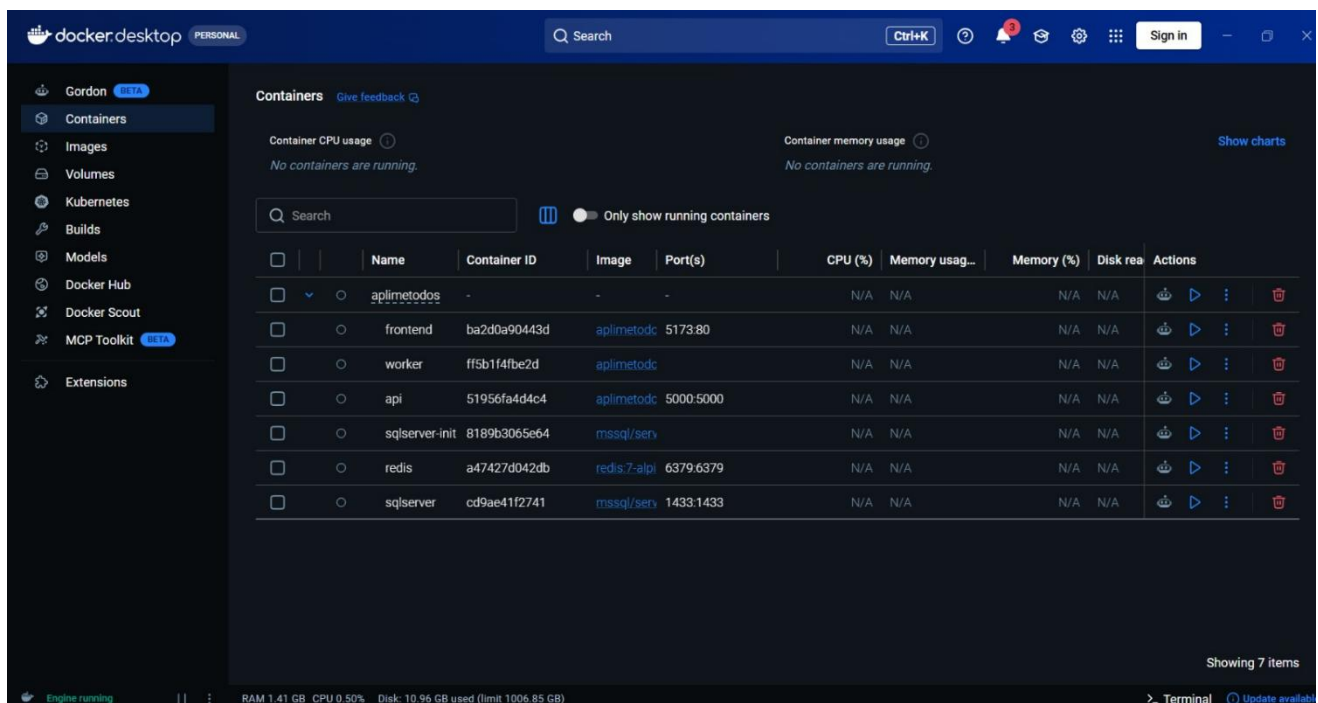
- Una vez abierto Docker Desktop, escribir el comando `docker compose up --build`.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\COMPRAS 2> cd "D:\Escritorio\EJEMPLO"
PS D:\Escritorio\EJEMPLO> git clone https://github.com/JoseAdrianMelgar/APLIMETODOS
Cloning into 'APLIMETODOS'...
remote: Enumerating objects: 220, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (31/31), done.
remote: Total 220 (delta 11), reused 6 (delta 3), pack-reused 186 (from 3)
Receiving objects: 100% (220/220), 89.73 MiB | 1.48 MiB/s, done.
Resolving deltas: 100% (76/76), done.
PS D:\Escritorio\EJEMPLO> cd APLIMETODOS
PS D:\Escritorio\EJEMPLO\APLIMETODOS> copy .env.example .env
PS D:\Escritorio\EJEMPLO\APLIMETODOS> docker compose up --build
```

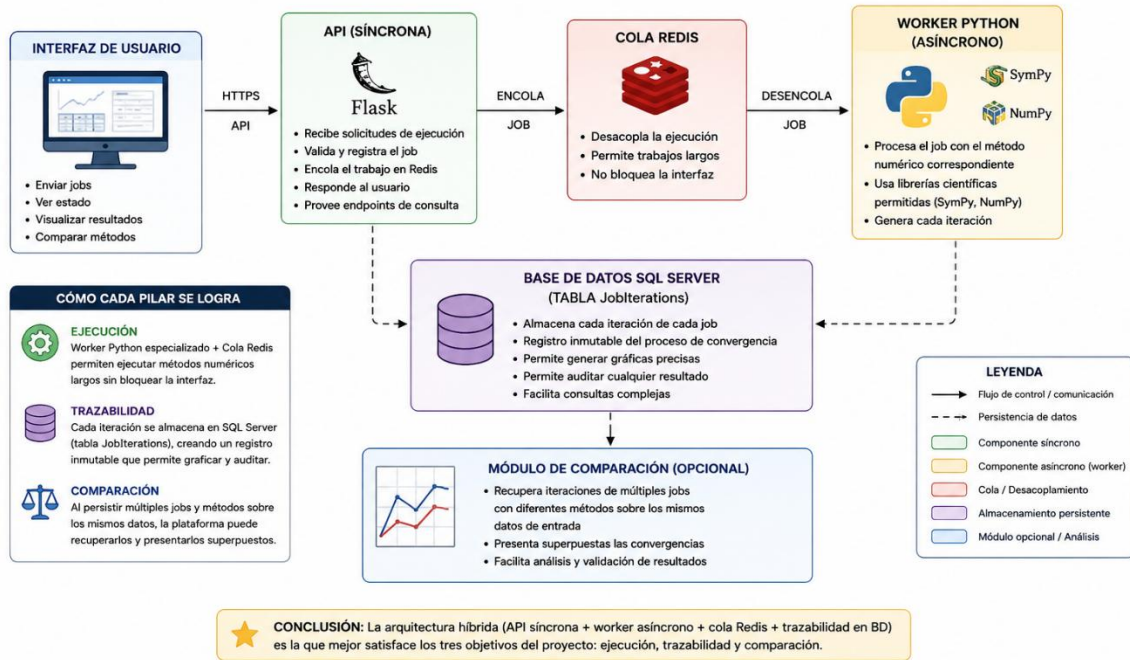
- Así se muestra ya levantado todo en Docker.
Luego se abre el front con el localhost que se muestra en la imagen.



1.3 Justificación de la arquitectura híbrida en el contexto del proyecto.

ARQUITECTURA DE LA PLATAFORMA

Tres pilares: ejecución, trazabilidad y comparación

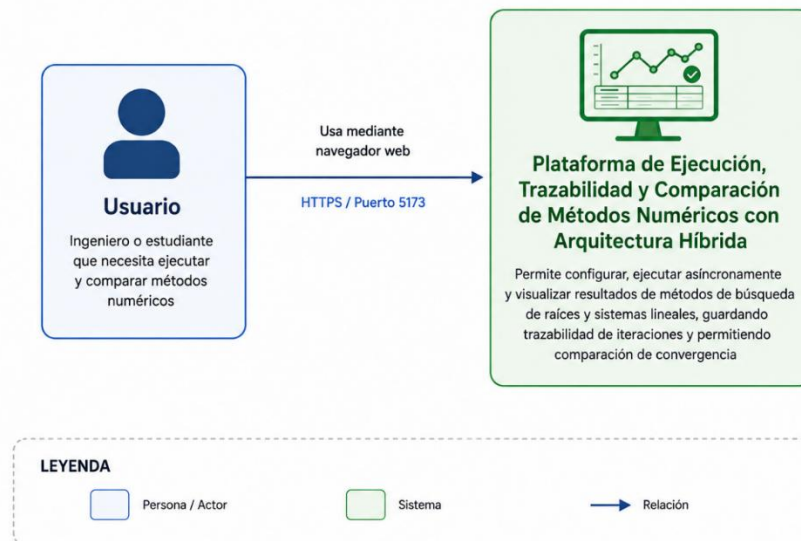


2. Diagrama C4 (Context + Container) o arquitectura del sistema

2.1 Diagrama de contexto

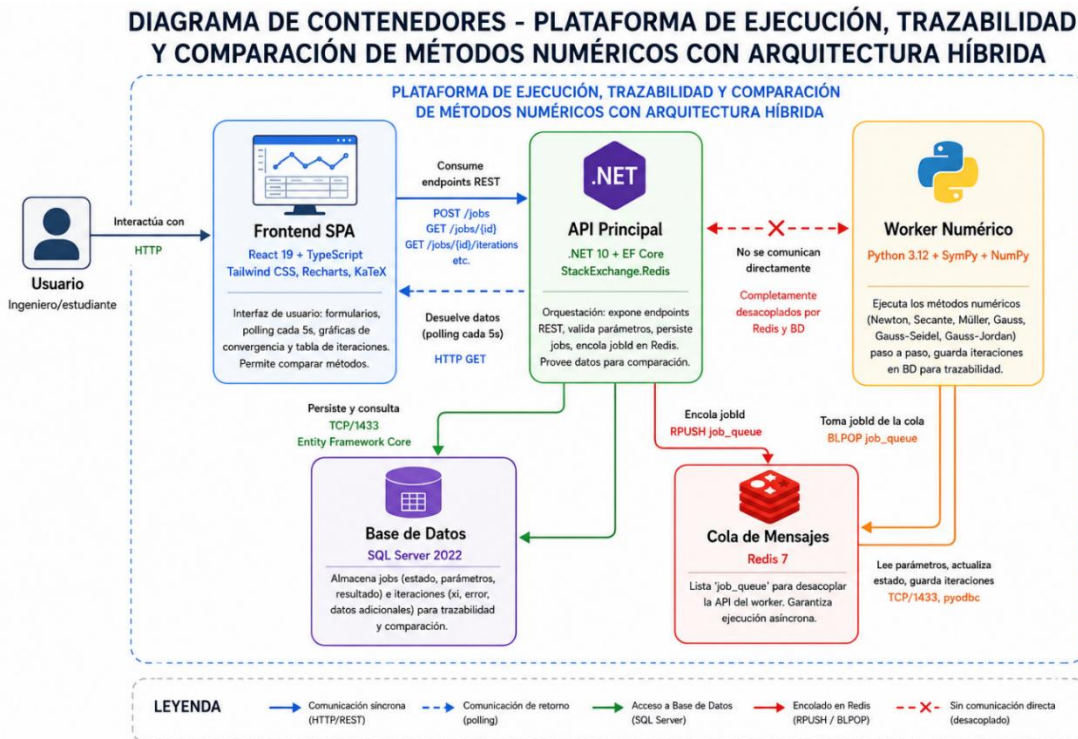
Muestra el sistema como una “Caja negra” y su interacción con el usuario externo.

DIAGRAMA DE CONTEXTO - PLATAFORMA DE EJECUCIÓN, TRAZABILIDAD Y COMPARACIÓN DE MÉTODOS NUMÉRICOS CON ARQUITECTURA HÍBRIDA



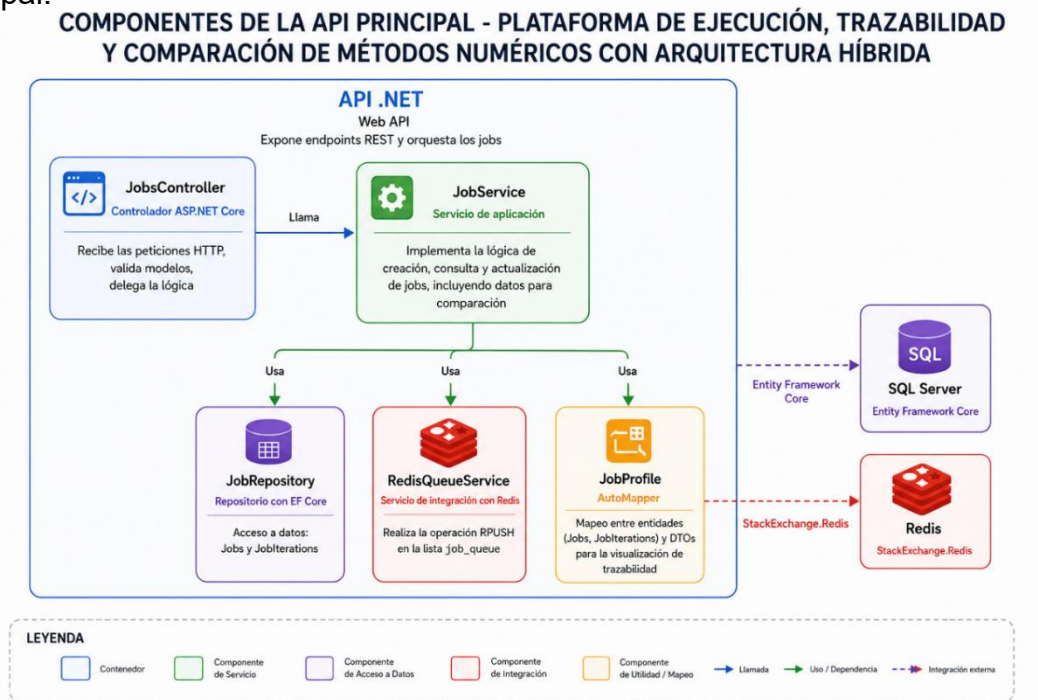
2.2 Diagrama de contenedores

Muestra los cinco servicios contenedorizados, sus tecnologías y los protocolos de comunicación.



2.3 Diagrama de Componentes (API .NET)

Para profundizar, se muestra el diagrama de componentes internos de la API principal.



2.4 Tabla de contenedores

Contenedor	Tecnología	Responsabilidad en la plataforma
 Frontend SPA	React 19 + TS	 UI para crear jobs, visualizar gráficas de convergencia, comparar métodos y ver trazabilidad de iteraciones.
 API .NET	.NET 10	 Endpoints REST, validación, persistencia de jobs y encolado asíncrono.
 Worker Python	Python 3.12	 Ejecución numérica con trazabilidad completa (cada iteración se guarda).
 Redis	Redis 7	 Cola de trabajos para ejecución asíncrona.
 SQL Server	SQL Server 2022	 Persistencia de jobs e iteraciones, permitiendo consultas históricas y comparativas.

3. Justificación de decisiones técnicas

3.1. ¿Por qué Redis?

- **Desacoplamiento real:** La API .NET y el Worker Python no necesitan conocerse entre sí ni estar en el mismo servidor. Redis actúa como intermediario.
- **Simplicidad:** La lista RPush / BLPOP es suficiente para este caso de uso (cola FIFO). No se necesita un sistema de mensajería más pesado como RabbitMQ.
- **Rendimiento:** Redis está en memoria, por lo que las operaciones de encolado son del orden de microsegundos, lo que evita cuellos de botella.
- **Idempotencia garantizada:** El Worker verifica el estado del job en BD antes de ejecutarlo, evitando reprocesar jobs ya **DONE** o **RUNNING**.

3.2 ¿Por qué Python para el worker?

- **Ecosistema matemático:** SymPy permite calcular derivadas simbólicas sin implementación manual; NumPy facilita **operaciones con matrices y vectores**.
- **Rapidez de desarrollo:** Python permite implementar los 6 métodos numéricos (Newton, Secante, Müller, Gauss, Gauss-Seidel, Gauss-Jordan) de forma legible y mantenible.
- **No se permite usar librerías que resuelvan el método completo:** Solo SymPy para derivadas y NumPy para álgebra lineal auxiliar. Toda la lógica iterativa está escrita manualmente.
- **Separación de preocupaciones:** La API (rápida, transaccional) se escribe en .NET; el cómputo pesado y científico en Python.

3.3 ¿Por qué Arquitectura Híbrida?

- **No bloqueo del usuario:** Si la ejecución de un método toma varios minutos, el navegador no se queda cargando indefinidamente. El usuario recibe el jobId inmediatamente y puede consultar el resultado más tarde.
- **Escalabilidad:** Se pueden ejecutar múltiples Workers Python en paralelo (replicando el contenedor) sin modificar la API ni Redis.
- **Trazabilidad completa:** Al guardar cada iteración en la base de datos, se puede generar la gráfica de convergencia y auditar el proceso.
- **Tolerancia a fallos:** Si un Worker falla, el job queda en estado **RUNNING**; se puede implementar un reintento o monitoreo sin perder datos.

4. Modelo de base de datos

4.1 Script de Creación Completo

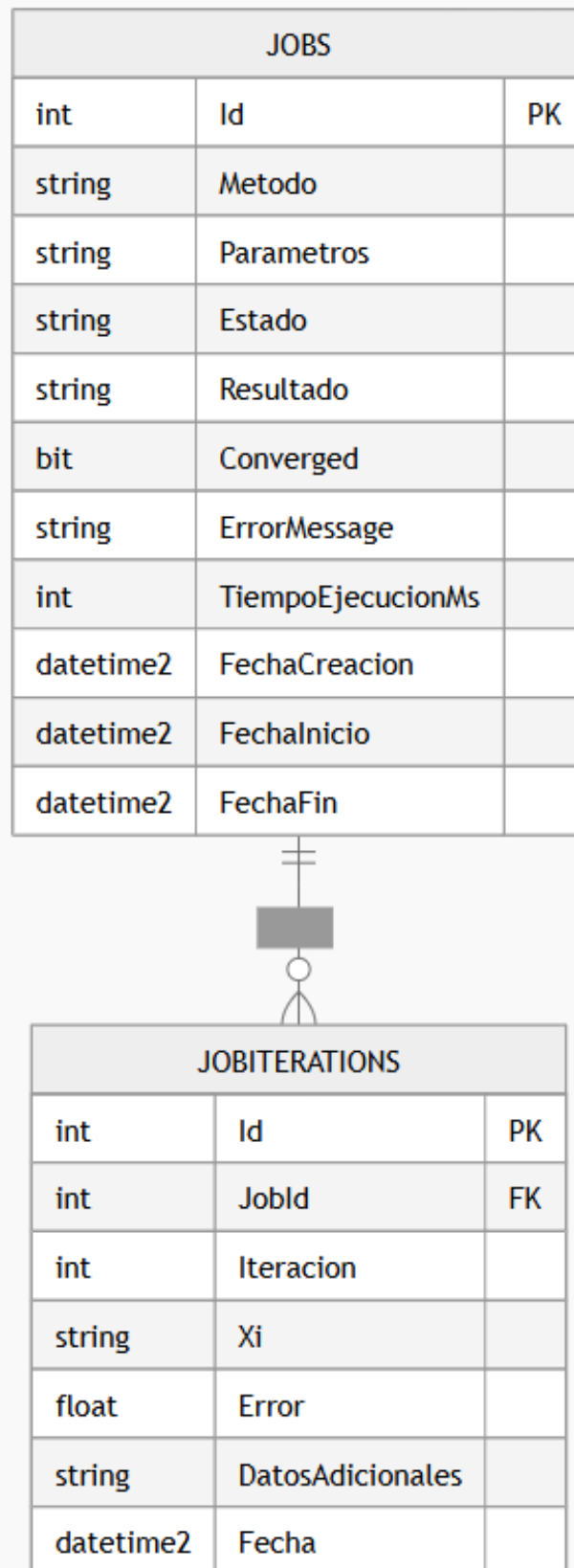
```
1  USE master;
2  GO
3
4  IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = N'MetodosNumericosDB')
5  BEGIN
6      CREATE DATABASE MetodosNumericosDB;
7  END
8  GO
9
10 USE MetodosNumericosDB;
11 GO
12
13 IF NOT EXISTS (SELECT * FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_NAME = 'Jobs')
14 BEGIN
15     CREATE TABLE Jobs (
16         Id INT IDENTITY(1,1) PRIMARY KEY,
17         Metodo NVARCHAR(50) NOT NULL,
18         Parametros NVARCHAR(MAX) NOT NULL,
19         Estado NVARCHAR(20) NOT NULL DEFAULT 'PENDING',
20         Resultado NVARCHAR(MAX) NULL,
21         Converged BIT NULL,
22         ErrorMessage NVARCHAR(MAX) NULL,
23         TiempoEjecucionMs INT NULL,
24         FechaCreacion DATETIME2 NULL DEFAULT GETDATE(),
25         FechaInicio DATETIME2 NULL,
26         FechaFin DATETIME2 NULL
27     );
28 END
29 GO
30
```

```

30
31 IF NOT EXISTS (SELECT * FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_NAME = 'JobIterations')
32 BEGIN
33     CREATE TABLE JobIterations (
34         Id INT IDENTITY(1,1) PRIMARY KEY,
35         JobId INT NOT NULL,
36         Iteracion INT NOT NULL,
37         Xi NVARCHAR(MAX) NULL,
38         Error FLOAT NULL,
39         DatosAdicionales NVARCHAR(MAX) NULL,
40         Fecha DATETIME2 NULL DEFAULT GETDATE(),
41         CONSTRAINT FK_JobIterations_Jobs FOREIGN KEY (JobId)
42             REFERENCES Jobs(Id) ON DELETE CASCADE
43     );
44
45     CREATE INDEX IX_JobIterations_JobId ON JobIterations(JobId);
46 END
47 GO
48
49 PRINT 'Base MetodosNumericosDB inicializada correctamente.';
50 GO

```

4.2 Diagrama Entidad-Relación



4.3 Tabla JOBS (Entidad Principal)

Campo	Tipo	¿Nulo?	Descripción
 Id	INT IDENTITY (1,1)	NO	Clave primaria. Se autoincrementa automáticamente.
 Método	NVARCHAR (50)	NO	Nombre del método numérico ejecutado.
 Parámetros	NVARCHAR (MAX)	NO	JSON con los parámetros de entrada (función, valor inicial, tolerancia, matriz, etc.).
 Estado	NVARCHAR (20)	NO	Estado actual del <code>job</code> . Valores posibles: PENDING, RUNNING, DONE, FAILED.
 Resultado	NVARCHAR (MAX)	SÍ	Raíz encontrada, vector solución o resultado final en formato texto.

 Converged	BIT	SÍ	Indica si el método convergió exitosamente (1) o no (0).
 ErrorMessage	NVARCHAR (MAX)	SÍ	Mensaje de error detallado cuando el job termina en FAILED.
 TiempoEjecucionMs	INT	SÍ	Duración total de la ejecución numérica, medida en milisegundos.
 FechaCreacion	DATETIME2	SÍ	Fecha y hora en que se creó el job (cuando el usuario hace POST).
 FechaInicio	DATETIME2	SÍ	Fecha y hora en que el worker comienza la ejecución (cambia a RUNNING).
 FechaFin	DATETIME2	SÍ	Fecha y hora en que el job termina (DONE o FAILED).

4.4 Tabla JOBITERATIONS (Trazabilidad)

Campo	Tipo	¿Nulo?	Descripción
 Id	INT IDENTITY(1,1)	NO	Clave primaria. Se autoincrementa automáticamente.
 JobId	INT	NO	Clave foránea que referencia a Jobs(Id) .
 Iteracion	INT	NO	Número de iteración (1, 2, 3 ...).
 Xi	NVARCHAR(MAX)	SÍ	Valor de la aproximación actual. Para raíces: número escalar. Para sistemas lineales: vector en formato JSON.
 Error	FLOAT	SÍ	Error relativo o absoluto calculado en esta iteración.
 DatosAdicionales	NVARCHAR(MAX)	SÍ	Información extra específica por método (ej: discriminante en Müller, matriz transformada en Gauss).
 Fecha	DATETIME2	SÍ	Marca de tiempo de cuando se insertó la iteración.

4.5 Relaciones y Restricciones

Relación llave foránea

FOREIGN KEY (JobId) REFERENCES Jobs(Id) ON DELETE CASCADE

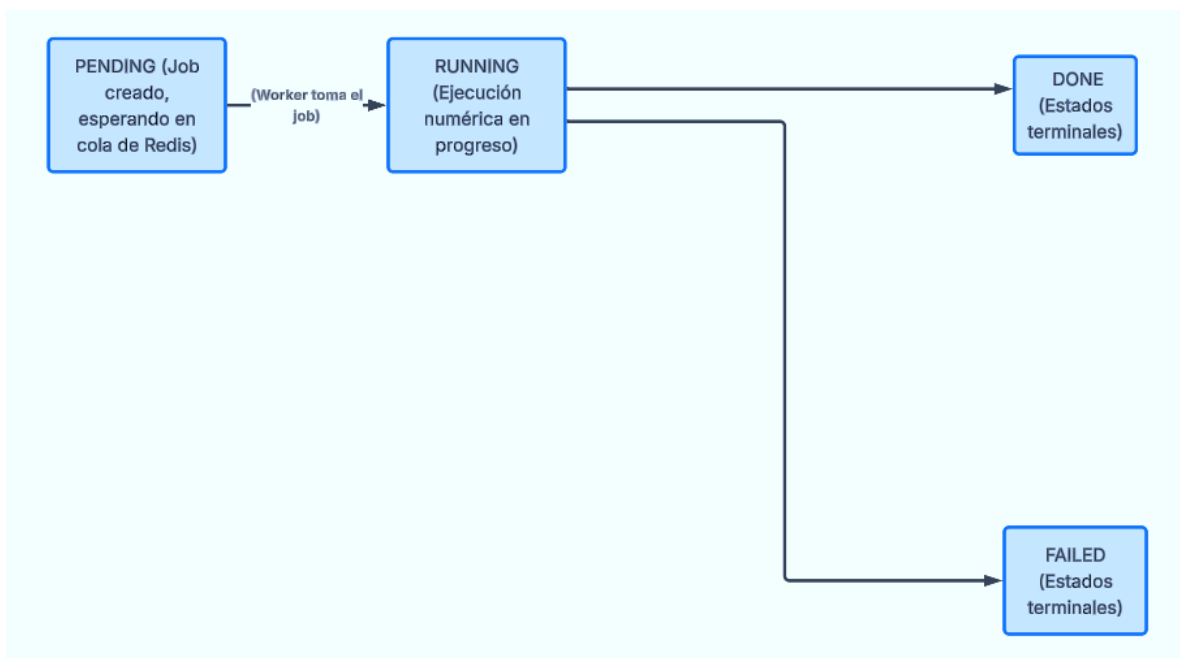
4.6 Índices para la optimización de rendimiento

Se crearon los siguientes índices para acelerar las consultas más frecuentes:

```
-- ÍNDICES  
CREATE INDEX IX_Jobs_Estado ON Jobs(Estado);  
CREATE INDEX IX_Jobs_FechaCreacion ON Jobs(FechaCreacion);  
CREATE INDEX IX_JobIterations_JobId ON JobIterations(JobId);  
GO
```

4.7 Ciclo de vida de los Estados

La columna Estado sigue estrictamente el siguiente flujo, garantizado por la API y el worker:



5. Instrucciones para clonar el repositorio

5.1 Herramientas que necesitaremos.

HERRAMIENTAS NECESARIAS

 HERRAMIENTA	 VERSIÓN MÍNIMA	 ¿PARA QUÉ SIRVE?
 Git	2.30+	Para clonar el repositorio desde GitHub. 
 Docker Desktop	20.10+	Para ejecutar los contenedores (incluye Docker Compose). 
 Navegador web	Actualizado	Para acceder al frontend (Chrome, Edge, Firefox). 

5.2 Instrucciones

Paso 1

Windows: Abre Git Bash (búscalo en el menú inicio).

Paso 2

Ejecutar el comando de clonación

git clone <https://github.com/JoseAdrianMelgar/APLIMETODOS.git>

Paso 3

Entrar a la carpeta del proyecto

cd APLIMETODOS

6. Guía de configuración de variables de entorno

La plataforma utiliza variables de entorno para parametrizar todos los servicios sin necesidad de modificar el código fuente. Esto permite cambiar la configuración (contraseñas, nombres de base de datos, hosts, puertos) de forma segura y sencilla, simplemente editando un archivo de texto. Además, evita que las credenciales sensibles se suban al repositorio de código.

Todas las variables se definen en un archivo llamado `.env` ubicado en la raíz del proyecto (junto al archivo `docker-compose.yml`). Docker Compose las lee automáticamente y las inyecta en cada contenedor.

✨ ✨ CONFIGURACIÓN DE VARIABLES DE ENTORNO ✨ ✨

 VARIABLE	 SERVICIO(S) QUE LA USAN	 PROPÓSITO	 VALOR DE EJEMPLO
 SQL_DATABASE	API .NET, Worker Python, SQL Server	Nombre de la base de datos que se creará y utilizará.	<code>MetodosNumericosDB</code>
 SQL_PASSWORD	API .NET, Worker Python, SQL Server	Contraseña del usuario administrador <code>sa</code> de SQL Server.	<code>Metodos2026!</code>
 REDIS_HOST	API .NET, Worker Python	Host o nombre del contenedor Redis dentro de la red Docker.	<code>redis</code>
 REDIS_PORT	API .NET, Worker Python	Puerto de conexión a Redis.	<code>6379</code>
 REDIS_QUEUE_NAME	API .NET, Worker Python	Nombre de la lista de Redis que se usa como cola de trabajos.	<code>job_queue</code>
 API_PORT (opcional)	API .NET	Puerto interno del contenedor de la API (y mapeo en host).	<code>5000</code>
 FRONTEND_PORT (opcional)	Frontend	Puerto interno del contenedor del frontend (y mapeo en host).	<code>5173</code>

6.1 Archivo de plantilla (.env.example)

El repositorio incluye un archivo de ejemplo llamado .env.example con la siguiente estructura:

```
SQL_DATABASE=MetodosNumericosDB
```

```
SQL_PASSWORD=Metodos2026!
```

6.2 Configuración Paso a Paso

- **Después de Clonar el repositorio**

```
git clone https://github.com/JoseAdrianMelgar/APLIMETODOS.git
cd APLIMETODOS
```

- **Copia la plantilla a un archivo llamado .env:**

```
cp .env.example .env
```

- **Edita el archivo .env con un editor de texto cambia al menos la contraseña por una segura:**

```
SQL_DATABASE=MetodosNumericosDB
SQL_PASSWORD=TuPasswordSeguro123!
```

- **Guarda el archivo. Docker Compose leerá automáticamente las variables definidas en .env al ejecutar docker compose up -- build.**

6.3 Como se inyectan las variables en cada servicio API .NET

En el docker-compose.yml, las variables se pasan al contenedor de la API mediante la sección environment:

```
api-dotnet:
  environment:
    - ASPNETCORE_ENVIRONMENT=Development
    - ConnectionStrings__DefaultConnection=Server=db;Database=${SQL_DATABASE};User Id=sa;
      Password=${SQL_PASSWORD};TrustServerCertificate=True;
    - Redis__Host=${REDIS_HOST}
    - Redis__Port=${REDIS_PORT}
    - Redis__QueueName=${REDIS_QUEUE_NAME}
```

Luego, en el código .NET (por ejemplo en appsettings.json o mediante IConfiguration), se accede a ellas con:

```
var connectionString = Configuration.GetConnectionString("DefaultConnection");  
var redisHost = Configuration["Redis:Host"];
```

Worker python

En el docker-compose.yml se pasan las mismas variables al Worker, y en Python se leen con `os.getenv()`:

```
import os  
db_name = os.getenv("SQL_DATABASE")  
db_password = os.getenv("SQL_PASSWORD")  
redis_host = os.getenv("REDIS_HOST", "localhost")  
redis_port = os.getenv("REDIS_PORT", "6379")
```

CONTENEDOR DE SQL SERVER

La variable `SA_PASSWORD` es obligatoria para la imagen oficial `mcr.microsoft.com/mssql/server`. En el `docker-compose.yml` se define así:

```
db:  
  environment:  
    - ACCEPT_EULA=Y  
    - SA_PASSWORD=${SQL_PASSWORD}  
    - MSSQL_PID=Express
```

7. Guía Paso a Paso para levantar el sistema con Docker

7.1 Requisitos previos

Docker Desktop instalado y abierto. Además, ya debes haber clonado el repositorio y configurado el archivo `.env`.

Paso 1

Asegúrate de estar en la carpeta correcta

```
cd APLIMETODOS
```

Paso 2

Construir y levantar los contenedores

```
docker compose up --build
```

Observaciones: La primera vez tardará de 5 a 15 minutos descargando imágenes. Y verás muchos mensajes en la terminal.

Paso 3

Espera a que todos los servicios estén listos

- db_1 | SQL Server 2022 will run as non-root...
- api-dotnet_1 | Now listening on: http://[::]:5000
- worker-python_1 | Worker started, waiting for jobs...
- frontend_1 | ready - started server

Paso 4

Abre el navegador y accede a la plataforma

<http://localhost:5173>

Paso 5

Probar un job de ejemplo.

- Selecciona Newton-Raphson.
- Función: $x^2 - 2$, x_0 : 1.0, tolerancia: 0.001, max iter: 50.
- Haz clic en Crear job.
- Espera a que termine (estado Done). Aparecerá la gráfica de convergencia y la tabla de iteraciones con la raíz ≈ 1.4142 .

Paso 6

Apagar el sistema

- Si ejecutaste `docker compose up` sin `-d`: presiona `Ctrl + C`.
- Si usaste `-d`: ejecuta `docker compose down`.

7.2 Solución a problemas comunes

Solución de problemas comunes

Problema	Posible causa	Solución
 db se reinicia en bucle	• Contraseña débil o no cumple política	 Edita <code>.env</code> con una contraseña fuerte y ejecuta <code>docker compose down -v</code>
 La API responde error 500	• Base de datos no creada	 Verifica que <code>SQL_DATABASE</code> coincida con <code>init.sql</code>
 El worker no consume jobs	• Redis no disponible	 Revisa <code>docker compose logs worker-python</code>
 El frontend no carga	• Puerto 5173 ocupado	 Cambia el puerto en <code>docker-compose.yml</code>

8. Pruebas realizadas y casos de error documentados con evidencia (Capturas o logs)

Durante las pruebas de estrés y validación realizadas sobre los métodos Newton-Raphson, Secante y Müller, se detectaron múltiples comportamientos incorrectos relacionados con convergencia falsa, bloqueo de Jobs, ausencia de validaciones, fallos del parser y colapso parcial del sistema bajo carga extrema.

Inconsistencia entre iteraciones y mensaje

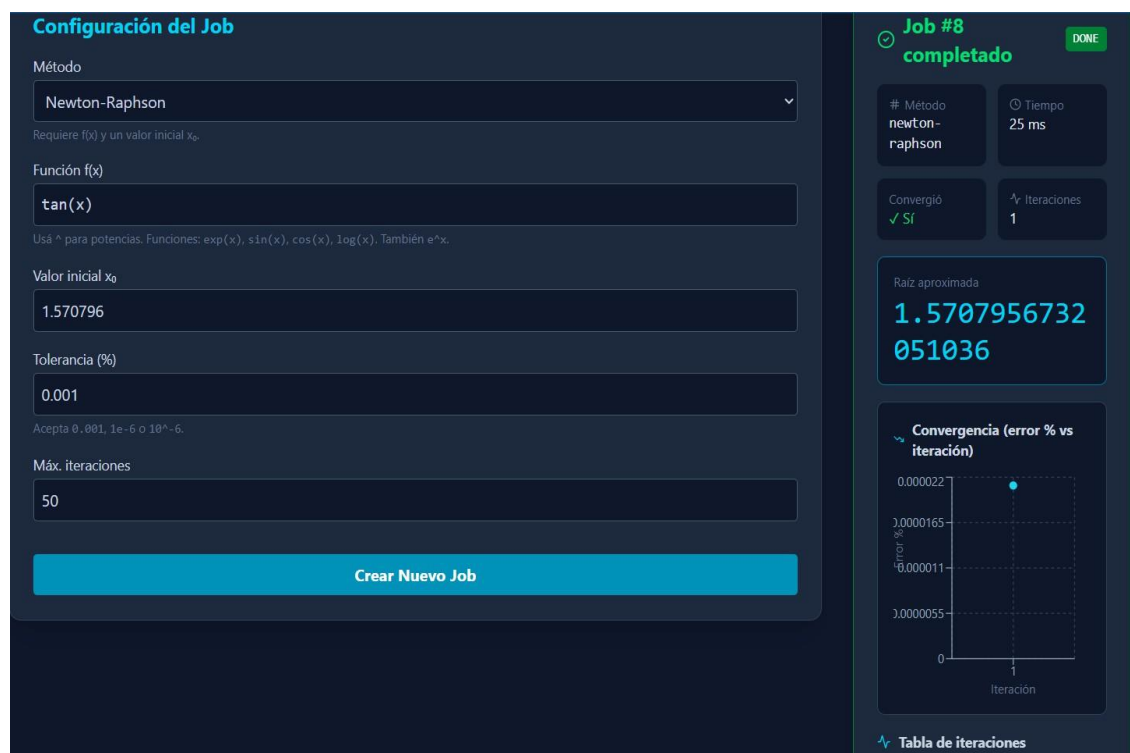
- El sistema indicó error en iteración 2, pero la tabla solo mostraba iteración 1.

Gráfica incompleta en fallos

- La gráfica no representó correctamente las iteraciones cuando el método falló.

Falsa convergencia con $\tan(x)$

- Newton-Raphson indicó convergencia cerca de $\pi/2$, aunque $\tan(x)$ tiene discontinuidad vertical.



Falta de validación en iteraciones máximas

El sistema aceptó cantidades absurdamente grandes de iteraciones y funciones lo cual genero un consumo grande, provocando problemas en el Sistema (recomendando colocar un límite o una advertencia)

[illegible]

Jobs colgados por spam

Al generar múltiples Jobs rápidamente, el programa se congelo y todos quedaron permanentemente en estado Pending.

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

xx

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

1

Acepta 0.001, 1e-6 o 10^-6.

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Procesando Job #103

Estado: Pending

Polling cada 5 segundos...

Ausencia de timeout

Los jobs largos no fueron cancelados automáticamente.

Pending infinito con $f(x)=x^{(1/3)}$

Newton-Raphson quedó permanentemente en Pending con comportamiento oscilante.

Pending con Müller y $\tan(x)$

El método Müller quedó colgado al trabajar con funciones discontinuas.

Colapso parcial del sistema

Después de múltiples pruebas extremas, el sistema presentó Network Error y dejó de responder correctamente.

The image shows two screenshots of a web application interface for configuring a job.

Top Screenshot: A dark-themed form titled "Configuración del Job". A modal error message is displayed: "localhost:5173 dice Error al crear el Job: Network Error" with an "Aceptar" button. The form fields include: "Método" (dropdown menu showing "Newton-Raphson"), "Requiere f(x) y un valor inicial x_0", "Función f(x)", "Valor inicial x_0", "Tolerancia (%)", and "Máx. iteraciones".

Bottom Screenshot: The same "Configuración del Job" form, but with a different function: "((((((((((x))))))))))". The "Valor inicial x_0" is set to "1", "Tolerancia (%)" is "0.001", and "Máx. iteraciones" is "50". A large blue button at the bottom says "Crear Nuevo Job". On the right side, a status panel shows a circular loading icon, the text "Procesando Job #104", and "Estado: Pending" with "Polling cada 5 segundos..." below it.